

DATA STRUCTURES & ALGORITHMS

Complete Mid-Term Preparation Guide

CSC211 · Fall 2025 · BCS/BSE

Topics Overview

Topic	Sub-Topics	Marks
Stack	Operations, Binary $\leftrightarrow$ Decimal, Infix $\rightarrow$ Postfix, Postfix Eval, Balanced Brackets	~25
Queue	Linear, Circular, Double-Ended (Deque)	~10
Linked List	Singly, Circular, Doubly + algorithms	~10
BST	Insert, Delete, Search, Traversals	~5
AVL Tree	Balance Factor, Height, Rotations (LL/RR/LR/RL)	~7
Heap Tree	Max/Min Heap, Deletion, Heap Sort, Priority Queue	~3

SECTION 1: STACK

1.1 Concept & Core Operations

A **Stack** is a linear data structure following **LIFO (Last In, First Out)** principle. Think of a pile of plates — you add/remove only from the top.

Push	Add element to top. Check overflow (size == MAX) first.
Pop	Remove & return top element. Check underflow (size == 0) first.
Peek/Top	Return top element without removing it.
isEmpty	Return true if size == 0.
isFull	Return true if size == MAX.

1.2 Binary ↔ Decimal Conversions using Stack

Decimal to Binary: Repeatedly divide by 2, push remainders onto stack, then pop to read the binary number MSB-first.

Step	Decimal	÷ 2	Remainder (Push)	Stack State
1	13	13÷2=6	1	[1]
2	6	6÷2=3	0	[1, 0]
3	3	3÷2=1	1	[1, 0, 1]
4	1	1÷2=0	1	[1, 0, 1, 1]
Pop all	—	—	—	Read: 1101

Binary to Decimal: Push binary digits, assign positional weights ($2^0, 2^1, 2^2, \dots$) from top to bottom while popping.

Example: 1101 → pop: $1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 8 + 4 + 0 + 1 = 13$

1.3 Infix to Postfix Conversion

Operator Precedence (High → Low):

- ^ (Exponent) → highest
- * / (Multiply, Divide)
- + - (Add, Subtract) → lowest

Algorithm:

- Scan expression left to right.
- If **operand** → output it directly.
- If '(' → push to stack.
- If ')' → pop and output until '(' is found; discard both brackets.
- If **operator** → pop & output operators with higher or equal precedence, then push current operator.
- At end → pop all remaining stack elements to output.

Full Worked Example: A+(B-C)*(D-E)+F/G*(H-J)

Symbol	Stack	Output	Action
A	[]	A	Operand → output
+	[+]	A	Push +

(	[+, (]	A	Push (
B	[+, (]	A B	Operand → output
-	[+, (, -]	A B	Push -
C	[+, (, -]	A B C	Operand → output
)	[+]	A B C -	Pop until (→ output -
*	[+, *]	A B C -	* > +, push *
(	[+, *, (]	A B C -	Push (
D	[+, *, (]	A B C - D	Operand → output
-	[+, *, (, -]	A B C - D	Push -
E	[+, *, (, -]	A B C - D E	Operand → output
)	[+, *]	A B C - D E -	Pop until (→ output -
+	[+]	A B C - D E - *+	Pop * and + (≥ prec), push +
F	[+]	A B C - D E - *+ F	Operand → output
/	[+, /]	A B C - D E - *+ F	/ > +, push /
G	[+, /]	A B C - D E - *+ F G	Operand → output
*	[+, *]	A B C - D E - *+ F G /	Pop / (= prec), push *
(	[+, *, (]	A B C - D E - *+ F G /	Push (
H	[+, *, (]	A B C - D E - *+ F G / H	Operand → output
-	[+, *, (, -]	A B C - D E - *+ F G / H	Push -
J	[+, *, (, -]	A B C - D E - *+ F G / H J	Operand → output
)	[+, *]	A B C - D E - *+ F G / H J -	Pop until (
END	[]	A B C - D E - *+ F G / H J - * +	Pop remaining

■ **KEY RULE:** When two operators have equal precedence, pop the existing one first (left-to-right associativity). Exception: ^ is right-associative.

1.4 Postfix Expression Evaluation

Algorithm: Scan left to right. If operand → push. If operator → pop two operands (second pop is left operand, first pop is right operand), apply operator, push result.

Example: Evaluate $5\ 3\ 2\ * + 4 -$

Symbol	Action	Stack After
5	Push 5	[5]
3	Push 3	[5, 3]
2	Push 2	[5, 3, 2]
*	Pop 2, 3 → $3*2=6$ → Push 6	[5, 6]
+	Pop 6, 5 → $5+6=11$ → Push 11	[11]
4	Push 4	[11, 4]
-	Pop 4, 11 → $11-4=7$ → Push 7	[7]
END	Result = top of stack	Answer: 7

1.5 Balanced Brackets / Parenthesis Checker

Algorithm:

- Scan each character in the string.
- If opening bracket ({ [→ push onto stack.
- If closing bracket) }] → if stack is empty → **UNBALANCED**. Otherwise pop and check if it matches the closing bracket.
- After full scan: if stack is empty → **BALANCED**, else → **UNBALANCED**.

Input	Stack State	Result
{[()]}	{ → [{ → [{(→ [{() pop } → [{ → { pop } → { → pop } →	BALANCED ✓
{[(]}	push {,[,(, see] but top is (→ MISMATCH	UNBALANCED ✗
((())	push (,(,(, pop),pop), stack has 2 left	UNBALANCED ✗

■ PRACTICE — Stack Questions (Past Paper Style)

Q1. [5 marks] Trace the queue code: enqueue(8), display, enqueue(3), display, enqueue(10), display, dequeue, display, enqueue(1), display, dequeue, display.

Q2. [10 marks] Convert to postfix showing stack state at each step: $A + ((B - C) * (D - E) + F) / G * (H - J)$

Q3. [5 marks] Evaluate postfix: $6\ 2\ 3\ +\ -\ 3\ 8\ 2\ /\ +\ * \ 2\ ^$

Q4. [5 marks] Check if the following are balanced: (a) {[()]} (b) [(]) (c) {{{[(())]]}}

SECTION 2: QUEUE

2.1 Linear Queue

A **Queue** follows **FIFO (First In, First Out)**. Elements are added at the **rear** and removed from the **front**. Implemented using an array of fixed size MAX.

Enqueue	Add element at rear. Check overflow ($\text{size} == \text{MAX}$). $\text{rear} = (\text{rear} + 1) \% \text{MAX}$; $\text{queue}[\text{rear}] = \text{item}$; $\text{size}++$
Dequeue	Remove from front. Check underflow ($\text{size} == 0$). $\text{temp} = \text{queue}[\text{front}]$; $\text{front} = (\text{front} + 1) \% \text{MAX}$; $\text{size}--$
Front Index	Points to the first element.
Rear Index	Points to the last inserted element.

2.2 Sorted Enqueue (Past Paper Code Analysis)

The code in your past paper implements a **Priority / Sorted Queue**. It inserts each new element in sorted (ascending) order by shifting larger elements to make room.

```
void enqueue(int number) {  
    int i;  
    for (i = size-1; i >= 0 && queue[i] > number; i--)  
        queue[i+1] = queue[i]; // shift right  
    queue[i+1] = number; // insert at correct position  
    size++;  
}
```

Trace the `main()` sequence:

Operation	Action	Queue Contents (front→rear)
<code>enqueue(8)</code>	Insert 8 (empty queue)	[8]
<code>displayQueue()</code>	Print	8
<code>enqueue(3)</code>	$3 < 8 \rightarrow$ shift 8 right $\rightarrow$ insert 3	[3, 8]
<code>displayQueue()</code>	Print	3 8
<code>enqueue(10)</code>	$10 > 8 \rightarrow$ append	[3, 8, 10]
<code>displayQueue()</code>	Print	3 8 10
<code>dequeue()</code>	Remove front (3), shift left	[8, 10]
<code>displayQueue()</code>	Print	8 10
<code>enqueue(1)</code>	$1 < 8 \rightarrow$ shift both right	[1, 8, 10]
<code>displayQueue()</code>	Print	1 8 10
<code>dequeue()</code>	Remove front (1)	[8, 10]
<code>displayQueue()</code>	Print	8 10

2.3 Circular Queue

Solves the **false overflow** problem of linear queue by wrapping around using modulo arithmetic.

- $\text{front} = (\text{front} + 1) \% \text{MAX}$ on dequeue
- $\text{rear} = (\text{rear} + 1) \% \text{MAX}$ on enqueue
- Full condition: $(\text{rear} + 1) \% \text{MAX} == \text{front}$

- Empty condition: $\text{front} == \text{rear}$ (or use a size counter)

Show contents using **Simple Queue notation**: $\text{front} \rightarrow \dots \rightarrow \text{rear}$

Show contents using **Circular Queue notation**: array indices $[0..\text{MAX}-1]$ with front/rear markers.

2.4 Double-Ended Queue (Deque)

A **deque** allows insertion and deletion at **both front and rear**. Implemented on an array of size 6 (indices 0–5).

EnqueueRear(x)	Add x at rear. $\text{rear} = (\text{rear}+1)\% \text{MAX}$; $\text{arr}[\text{rear}] = x$;
EnqueueFront(x)	Add x at front. $\text{front} = (\text{front}-1+\text{MAX})\% \text{MAX}$; $\text{arr}[\text{front}] = x$;
DequeFront()	Remove from front. $\text{temp} = \text{arr}[\text{front}]$; $\text{front} = (\text{front}+1)\% \text{MAX}$;
DequeRear()	Remove from rear. $\text{temp} = \text{arr}[\text{rear}]$; $\text{rear} = (\text{rear}-1+\text{MAX})\% \text{MAX}$;

Past Paper Deque Trace — Array size 6, initial $\text{front} = \text{rear} = -1$ (empty):

Sequence: EnqueueRear(10), EnqueueRear(20), EnqueueFront(5), EnqueueRear(30), DequeueFront(), EnqueueRear(10), EnqueueRear(50), EnqueueFront(2), DequeueRear(), EnqueueFront(1)

Operation	Front Idx	Rear Idx	Array [0..5]	Show Contents
EnqueueRear (10)	0	0	[10, _, _, _, _, _]	F=10=R
EnqueueRear (20)	0	1	[10, 20, _, _, _, _]	10→20
EnqueueFront (5)	5	1	[10, 20, _, _, _, 5]	5→10→20
EnqueueRear (30)	5	2	[10, 20, 30, _, _, 5]	5→10→20→30
DequeFront ()	0	2	[10, 20, 30, _, _, _]	10→20→30 (5 removed)
EnqueueRear (10)	0	3	[10, 20, 30, 10, _, _]	10→20→30→10
EnqueueRear (50)	0	4	[10, 20, 30, 10, 50, _]	10→20→30→10→50
EnqueueFront (2)	5	4	[10, 20, 30, 10, 50, 2]	2→10→20→30→10→50
DequeRear ()	5	3	[10, 20, 30, 10, _, 2]	10→20→30→10 (50 removed)
EnqueueFront (1)	4	3	[10, 20, 30, 10, 1, 2]	1→2→10→20→30→10

■ PRACTICE — Queue Questions

Q1. [5 marks] Trace a linear queue of size 5: Enqueue(12), Enqueue(7), Enqueue(3), Dequeue(), Enqueue(15), Enqueue(9), show state after each step.

Q2. [10 marks] Implement a Circular Queue of size 5. Trace: Enqueue(A), Enqueue(B), Enqueue(C), Dequeue(), Dequeue(), Enqueue(D), Enqueue(E), Enqueue(F). Show front/rear after each op.

Q3. [10 marks] Trace the deque (size 6): EnqueueRear(5), EnqueueRear(10), EnqueueFront(3), EnqueueRear(15), DequeueFront(), DequeueRear(), EnqueueFront(1). Show front/rear/contents.

SECTION 3: LINKED LIST

3.1 Singly Linked List

Each node contains **data** and a **next** pointer. The last node's next = NULL. Head points to the first node.

Insert at Head	newNode.next = head; head = newNode;
Insert at Tail	Traverse to last node; last.next = newNode; newNode.next = NULL
Insert at Pos k	Traverse to (k-1)th node; newNode.next = prev.next; prev.next = newNode
Delete Head	head = head.next;
Delete by Value	Traverse to find node; prev.next = curr.next;
Search	Traverse while curr != NULL, check curr.data == target

3.2 Past Paper — LinkedList Rotation by k

The code rotates a linked list by k positions. Specifically, it makes the list start from the $(k \% \text{length})$ th node:

```
static Node LinkedList(Node head, int k) {  
    // 1. Find length  
    Node current = head; int length = 1;  
    while (current.next != null) { current = current.next; length++; }  
    // 2. k = k % length (handle k > length)  
    k = k % length;  
    if (k == 0) return head;  
    // 3. Connect tail to head (make circular)  
    current.next = head;  
    // 4. Find new tail = (length - k - 1) steps from head  
    for (int i = 0; i < length - k; i++) current = current.next;  
    // 5. New head is current.next; break circle  
    Node newHead = current.next;  
    current.next = null;  
    return newHead;  
}
```

List: 1→2→3→4→5→6→7, length = 7

k value	k % 7	New Head index (1-based)	Resulting List
k=4	4%7=4	Node at pos $(7-4+1)=4 \rightarrow$ node 4	4→5→6→7→1→2→3
k=8	8%7=1	Node at pos $(7-1+1)=7 \rightarrow$ node 7	7→1→2→3→4→5→6

3.3 Circular Linked List

The last node's **next** points back to the **head** instead of NULL. Traversal must stop when `current.next == head`.

- Insert: `newNode.next = head; last.next = newNode; head = newNode;` (update last's pointer)
- Delete head: `last.next = head.next; head = head.next;`

3.4 Doubly Linked List

Each node has **prev**, **data**, and **next** pointers. Allows traversal in both directions.

Insert at Head	<code>newNode.next = head; head.prev = newNode; newNode.prev = NULL; head = newNode;</code>
Insert at Tail	<code>last.next = newNode; newNode.prev = last; newNode.next = NULL;</code>
Delete a node	<code>node.prev.next = node.next; node.next.prev = node.prev;</code>
Merge 2 lists	Algorithm: compare heads, attach smaller, advance that pointer, repeat.

3.5 Merge Two Sorted Linked Lists (Past Paper Q2A)

Algorithm:

- Create a dummy head node to simplify merging logic.
- Use a 'current' pointer starting at dummy.
- While both lists are non-null: compare list1.data and list2.data. Attach the smaller node to current.next. Advance that list's pointer. Advance current.
- Append the remaining non-null list to current.next.
- Return dummy.next as the merged list head.

Example: list1 = 10→30→50, list2 = 20→40→60→80

Step	list1 ptr	list2 ptr	Merged so far
1	10	20	10
2	30	20	10→20
3	30	40	10→20→30
4	50	40	10→20→30→40
5	50	60	10→20→30→40→50
6	NULL	60	10→20→30→40→50→60→80 (append rest)

■ PRACTICE — Linked List Questions

Q1. [10 marks] Write a stepwise algorithm to merge two sorted linked lists. Use list1 = 5→15→25, list2 = 10→20→30. Show each merge step.

Q2. [10 marks] Apply the rotation code on list [1-2-3-4-5-6-7] for k=3 and k=10. Show intermediate steps.

Q3. [5 marks] In a doubly linked list 1↔2↔3↔4↔5, delete node 3. Show prev/next pointer updates.

SECTION 4: BINARY SEARCH TREE (BST)

4.1 BST Properties

- Left subtree of any node contains only values **less than** the node.
- Right subtree of any node contains only values **greater than** the node.
- Both left and right subtrees are also BSTs.
- No duplicate values.

4.2 BST Operations

Insert	Start at root. Go left if value < node, right if value > node. Insert at NULL position.
Search	Same traversal as insert. Return true if found, false if NULL reached.
In-order	Left → Root → Right. Produces sorted output.
Pre-order	Root → Left → Right. Used to copy/serialize a tree.
Post-order	Left → Right → Root. Used to delete/evaluate a tree.

4.3 Past Paper BST — Insert & Delete

Values to insert in order: 50, 30, 70, 20, 40, 60, 80, 35, 45, 75, 85

Resulting BST structure:

```
50
 / \
30 70
 / \ / \
20 40 60 80
 / \ \ \
35 45 75 85
```

Delete node 70 (node with TWO children):

- Find 70's **in-order successor**: smallest node in right subtree → go right to 80, then left → 75. In-order successor = **75**.
- Replace 70's value with 75.
- Delete 75 from its original position (it has no left child → simple deletion).

Tree after deleting 70 (replaced by 75):

```
50
 / \
30 75
 / \ / \
20 40 60 80
 / \ \ \
35 45 85
```

Delete leaf node	Simply remove it (set parent's pointer to NULL).
Delete node w/ 1 child	Replace node with its child.
Delete node w/ 2 children	Replace value with in-order successor (or predecessor), delete the successor.

■ PRACTICE — BST Questions

Q1. [3 marks] Insert 50, 30, 70, 20, 40, 60, 80, 35, 45, 75, 85 into an empty BST. Draw the resulting tree.

Q2. [5 marks] Delete node 70 from the above BST. Show the tree after deletion with all pointer updates.

Q3. [3 marks] Write the in-order, pre-order, and post-order traversal of the BST from Q1.

SECTION 5: AVL TREE

5.1 AVL Tree Concepts

An **AVL Tree** is a self-balancing BST where the heights of left and right subtrees of **any** node differ by at most **1**.

Balance Factor (BF)	$BF(node) = Height(Left\ subtree) - Height(Right\ subtree)$
Valid BF values	-1, 0, or +1. If $ BF > 1 \rightarrow$ tree is unbalanced $\rightarrow$ rotation needed.
Height of node	$Height = 1 + \max(Height(left), Height(right))$. Height of NULL = -1 (or 0 by convention).
BF = +1	Left-heavy (left subtree is taller).
BF = -1	Right-heavy (right subtree is taller).
BF = 0	Perfectly balanced.

5.2 Four Rotation Types

Rotation	Trigger Condition	How to Rotate	Memory Tip
LL (Right Rotation)	BF = +2 at node Z, BF = +1 or 0 at left child Y	Rotate Z to the right: Y becomes new root. Z becomes Y's right child. Y's right child becomes Z's left child.	Imbalance on Left-Left side $\rightarrow$ Right Rotate
RR (Left Rotation)	BF = -2 at node Z, BF = -1 or 0 at right child Y	Rotate Z to the left: Y becomes new root. Z becomes Y's left child. Y's left child becomes Z's right child.	Imbalance on Right-Right side $\rightarrow$ Left Rotate
LR (Left-Right)	BF = +2 at node Z, BF = -1 at left child Y	Step 1: Left rotate Y (makes it LL case). Step 2: Right rotate Z.	Left then Right rotation (double)
RL (Right-Left)	BF = -2 at node Z, BF = +1 at right child Y	Step 1: Right rotate Y (makes it RR case). Step 2: Left rotate Z.	Right then Left rotation (double)

5.3 Full AVL Insertion Trace (Past Paper Q3C)

Insert sequence: 50, 30, 70, 20, 40, 35, 80

Insert	Tree State (simplified)	BF at nodes	Rotation?
50	50 (root)	BF(50)=0	None
30	50 └─L:30	BF(50)=+1, BF(30)=0	None
70	50 ├─30 70	BF(50)=0, BF(30)=0, BF(70)=0	None
20	50 ├─30 70 └─20	BF(50)=+1, BF(30)=+1, BF(20)=0	None
40	50 ├─30 70 └─20 40	BF(50)=+1, BF(30)=0	None

35	35 inserted under 40 as left child. BF(40)=+1, BF(30)=-1 → check 50: BF=+2 Left child 30: BF=-1 → LR ROTATION!	BF(50)=+2 UNBALANCED	LR Rotation at 50: 1) Left rotate 30 2) Right rotate 50 New root: 40
80	Insert 80 as right child of 70. All BF's: 0 or ±1	BF(40)=0, all balanced	None

Final AVL Tree after all insertions (after LR rotation at 50):

```

40
/  \
30  50
/  \  \
20  35 70
\
80

```

■ **Always check BF's bottom-up after each insertion. Fix the FIRST unbalanced ancestor.**

■ PRACTICE — AVL Tree Questions

Q1. [7 marks] Insert [50, 30, 70, 20, 40, 35, 80] into an empty AVL tree step by step. Show the tree and BF after each insertion, perform rotations as needed.

Q2. [5 marks] Insert [10, 20, 30, 40, 50] into an empty AVL tree. This triggers multiple RR rotations — show each.

Q3. [5 marks] Insert [30, 20, 40, 10, 25, 35, 50, 5, 15, 28, 27] into AVL tree. Identify all rotation types needed.

SECTION 6: HEAP TREE, HEAP SORT & PRIORITY QUEUE

6.1 Heap Tree Concepts

A **Heap** is a **Complete Binary Tree** that satisfies the **Heap Property**:

- **Max Heap**: Every parent's value $\geq$ its children's values. Root = Maximum element.
- **Min Heap**: Every parent's value $\leq$ its children's values. Root = Minimum element.

Array representation: For node at index i :

- Parent: $(i-1)/2$
- Left child: $2i+1$
- Right child: $2i+2$

6.2 Heapify / Bubble-Up (Insertion)

To insert a new element:

- Add at end of array (last position in complete tree).
- Compare with parent. If heap property violated, swap.
- Repeat ('bubble up') until heap property is satisfied or root is reached.

Example — Insert 35 into Max-Heap [80, 70, 60, 50, 40, 30, 20]:

Step	Action	Array State
1	Add 35 at end (index 7)	[80, 70, 60, 50, 40, 30, 20, 35]
2	Parent of 7 = $(7-1)/2 = 3 \rightarrow$ value 50. $35 < 50 \rightarrow$ OK	No swap needed
Result	Heap valid	[80, 70, 60, 50, 40, 30, 20, 35]

6.3 Heap Deletion (Extract Root)

To delete the root (max/min):

- Replace root with the **last element**.
- Remove the last element.
- 'Heapify down' (sift down): Compare new root with its children. Swap with the **larger child** (Max-Heap) until heap property holds.

Example — Delete root from Max-Heap [80, 70, 60, 50, 40, 30, 20]:

Step	Action	Array State
1	Remove 80, move last (20) to root	[20, 70, 60, 50, 40, 30]
2	Root=20, children=70,60. Max child=70. $20 < 70 \rightarrow$ swap	[70, 20, 60, 50, 40, 30]
3	At idx1=20, children=50,40. Max=50. $20 < 50 \rightarrow$ swap	[70, 50, 60, 20, 40, 30]
4	At idx3=20, children=none beyond size. STOP	[70, 50, 60, 20, 40, 30] ✓

6.4 Heap Sort

Heap Sort uses a Max-Heap to sort an array in ascending order:

- **Phase 1 — Build Max-Heap**: Heapify from the last non-leaf node $(n/2-1)$ down to 0.
- **Phase 2 — Extract**: Repeatedly swap root (max) with last unsorted element, reduce heap size by 1, heapify root.

Time Complexity: $O(n \log n)$ — best, average, and worst case.

Space Complexity: $O(1)$ — in-place sort.

Example — Sort [4, 10, 3, 5, 1]:

Phase	Action	Array State
Build Heap	Heapify from idx 1 downward	[10, 5, 3, 4, 1]
Extract 1	Swap root(10) with last(1), heapify [4]	[5, 4, 3, 1, 10]
Extract 2	Swap root(5) with last(1), heapify	[4, 1, 3, 5, 10]
Extract 3	Swap root(4) with last(3), heapify	[3, 1, 4, 5, 10]
Extract 4	Swap root(3) with last(1)	[1, 3, 4, 5, 10]
Done	Sorted ascending	[1, 3, 4, 5, 10] ✓

6.5 Priority Queue using Heap

A **Priority Queue** is an ADT where each element has a priority. Elements are served based on priority, not insertion order.

- Max Priority Queue → Max Heap (highest priority served first).
- Min Priority Queue → Min Heap (lowest value = highest priority).

Enqueue(item, priority)	Insert item. Bubble up to restore heap property. $O(\log n)$
Dequeue()	Remove and return root (highest priority). Heapify down. $O(\log n)$
Peek()	Return root without removing. $O(1)$
Real-world uses	OS process scheduling, Dijkstra's shortest path, A* search, Huffman coding, emergency room triage

6.6 Min/Max Heap — Comparison Table

Property	Max Heap	Min Heap
Root contains	Maximum element	Minimum element
Parent vs Children	Parent $\geq$ Children	Parent $\leq$ Children
Extract gives	Largest element first	Smallest element first
Heap Sort result	Ascending order	Descending order
Priority Queue type	Highest value = highest priority	Lowest value = highest priority

■ PRACTICE — Heap & Sorting Questions

- Q1. [5 marks] Build a Max-Heap from the array [3, 1, 6, 5, 2, 4]. Show the tree after each insertion using bubble-up.
- Q2. [5 marks] Delete the root from Max-Heap [90, 80, 70, 60, 50, 40, 30]. Show heapify-down step by step.
- Q3. [7 marks] Sort [12, 11, 13, 5, 6, 7] using Heap Sort. Show the max-heap build phase and each extraction step.
- Q4. [3 marks] Explain how a priority queue is implemented using a heap. What is the time complexity of enqueue and dequeue?

SECTION 7: PRACTICE EXAMINATION (Past Paper Pattern)

Mid-Term Practice Paper — Data Structures (CSC211)

Time Allowed: 1 Hour 30 Minutes | Maximum Marks: 60

Instructions: All questions are compulsory. Show all work for full marks.

Question 1 (25 Marks)

[Employ linear data structures to solve computing problems]

Part A — [5 Marks]

Read the code carefully and analyze the following operations. After each operation, state the current contents of the queue. The queue uses sorted enqueue (ascending order), $MAX = 5$.

Operations: enqueue(15), displayQueue(), enqueue(5), displayQueue(), enqueue(20), displayQueue(), dequeue(), displayQueue(), enqueue(10), displayQueue(), dequeue(), displayQueue()

Part B — [10 Marks]

Trace the steps involved in converting the following infix expression to postfix notation using a stack. For each step, clearly show the contents of the stack, and indicate when operators and brackets are pushed or popped.

$(A+B)*(C-D)/(E+F)-G*H$

Part C — [10 Marks]

Perform the following sequence of operations on a Double-Ended Queue (deque) implemented using an array of size 6 (indices 0–5). Show the state of the deque and the front/rear index positions after each operation.

EnqueueRear(8), EnqueueRear(16), EnqueueFront(4), EnqueueRear(24), DequeueFront(), EnqueueRear(12), EnqueueRear(32), EnqueueFront(2), DequeueRear(), EnqueueFront(1)

Question 2 (20 Marks)

[Employ linear data structures to solve computing problems]

Part A — [10 Marks]

You are given two sorted singly linked lists. Merge them into a new sorted linked list maintaining sorted order.

list1 = $2 \rightarrow 5 \rightarrow 8 \rightarrow 11$

list2 = $1 \rightarrow 4 \rightarrow 7 \rightarrow 10 \rightarrow 13$

Write a stepwise algorithm and trace each merge step. Show the merged list.

Part B — [10 Marks]

Apply the LinkedList rotation function on the singly linked list [1–2–3–4–5–6–7–8] with head at node 1. What would be the output linked lists for the values of $k = 6$ and $k = 10$? Show all intermediate steps including length calculation, new head computation, and final list.

Question 3 (15 Marks)

[Use non-linear data structures to solve computing problems]

Part A — [3 Marks]

Insert the following values in the given order into an initially empty Binary Search Tree (BST):

40, 20, 60, 10, 30, 50, 70, 25, 35, 65, 75

Draw the resulting BST.

Part B — [5 Marks]

From the BST constructed in Part A, delete the node with value '60'. Clearly show:

- Which node replaces 60 (in-order successor or predecessor).
- The updated tree structure after deletion.

Part C — [7 Marks]

You are given the following sequence of integers to be inserted in order into an initially empty AVL Tree:

[10, 20, 30, 15, 25, 5, 35]

Construct the resulting AVL Tree, showing the tree after each insertion. You must perform rotations whenever required. Clearly show the Balance Factor of each node after each step.

Quick Reference — Key Formulas & Facts

Concept	Formula / Key Fact
Stack (LIFO)	Push: $top++$; $arr[top]=x$ Pop: $x=arr[top]$; $top--$
Queue (FIFO)	Enqueue at rear, Dequeue from front Circular: $(i+1)\%MAX$
Balance Factor	$BF = H(left) - H(right)$ Valid: $\{-1, 0, +1\}$
Height of node	$H = 1 + \max(H(left), H(right))$ $H(null) = -1$
Heap parent/child	$Parent=(i-1)/2$ $Left=2i+1$ $Right=2i+2$
Heap Sort complexity	$O(n \log n)$ time $O(1)$ space In-place, not stable
Infix→Postfix prec.	$^ (3) > */ (2) > +- (1)$ '(' pushed, ')' pops till '('
Postfix evaluation	Operand→push Operator→pop 2, compute, push result
BST Delete 2-child	Replace with in-order SUCCESSOR (leftmost of right subtree)
AVL LR rotation	Left rotate child, then right rotate grandparent
AVL RL rotation	Right rotate child, then left rotate grandparent

Good luck on your exam! Remember: Show your work, trace step-by-step, and draw trees clearly!